Lexical and Syntax Analyzers

CS3361 – Project #1 and #2

Dr. Eric Rees
Spring 2026

Table of Contents

Introduction 3
Purpose of the Projects 3
How These Projects Will Help You Understand Compilers 3
Educational Outcomes 3
The Cooke Programming Language 4
Introduction to the Cooke Programming Language 4
Backus-Naur Form (BNF) Rules for the Cooke Programming Language 4
Lexemes for the Cooke Programming Language 5
Project #1 – Lexical Analyzer 6
Problem 6
Project Requirements 6
AI Usage Policy 7
Hints 7
What to turn in to RaiderCanvas 7
Example Execution 8
Project #2 – Syntax Parser 9
Problem Statement 9
Project Requirements 9
AI Usage Policy 10
Hints 11
What to submit to RaiderCanvas 11
Example Execution 11
Optional Extra Credit 12
Project #2: Finding all syntax errors in a file (+15) 12
Requirements for Extra Credit Consideration 12
Important Warning 12
Appendix A – AI Reflection Report Requirements 13
Purpose 13
Submission Details 13
Report Layout 13

AI Reporting Guidelines 13
Section 1: Authorship Statement 14
Section 2: AI Chat Logs 15
Report Requirements 15
Verification and Responsibility 15
Authenticity and Correlation 15
Professionalism and Format 15
Academic Integrity Reminder 16

Introduction

Below is the description for Projects #1 and #2 for CS 3361 where you will delve into the essential components of compiler design by constructing a Lexical Analyzer and a Syntax Parser for the "Cooke" programming language. These projects are meticulously crafted to provide you with practical experience in the early stages of the compilation process, bridging the gap between the theoretical concepts we covered in class and real-world application.

Purpose of the Projects

The primary aim of these projects is to enhance your understanding of how compilers transform high-level programming code into machine-readable instructions. By engaging in the development of a lexical analyzer and a syntax parser, you will explore the critical processes of tokenization and grammatical validation, which are integral to compiler functionality.

How These Projects Will Help You Understand Compilers

1) **Lexical Analyzer (Project #1):**
   - **Tokenization**: Learn how source code is scanned and divided into a sequence of tokens, such as keywords, identifiers, operators, and literals.
   - **Pattern Recognition**: Implement algorithms to recognize and categorize different lexemes based on predefined patterns and rules.
   - **Error Handling**: Understand how to detect and handle invalid tokens, enhancing the robustness of the lexical analyzer.
2) **Syntax Analyzer (Project #2):**
   - **Parsing Techniques**: Apply recursive descent parsing to interpret the structure of the tokenized code according to the Cooke language grammar.
   - **Validation**: Ensure that the code adheres to grammatical rules, identifying and reporting syntax errors effectively.

Educational Outcomes

By completing these projects, you will:

- **Gain Practical Skills**: Develop hands-on experience in writing components that are critical to compiler operations.
- **Understand Compiler Architecture**: Build a foundational knowledge of how compilers work internally, preparing you for more advanced topics like semantic analysis and code optimization.
- **Enhance Problem-Solving Abilities**: Tackle common challenges in compiler construction, such as ambiguity resolution and efficient error reporting.
- **Connect Theory with Practice**: Apply theoretical concepts from lectures to practical coding tasks, solidifying your understanding through implementation.

These projects are not just academic exercises; they are the essential first steps toward mastering the complexities of compiler design. By the end of this journey, you will have a deeper appreciation for the intricacies involved in translating human-readable code into executable programs, a skill set highly valued in the field of computer science and software engineering.

The Cooke Programming Language

Introduction to the Cooke Programming Language

The **Cooke Programming Language** is a small imperative language designed with syntax reminiscent of Pascal, C, C++, and other structured programming languages, yet distinct in its own right. It serves as a compact study in language design, demonstrating how simplicity and clarity can coexist within a rigorously defined grammar.

The language is named in honor of Dr. Daniel E. Cooke, a distinguished computer scientist, language theorist, and mentor whose career embodied both intellectual depth and a passion for teaching. As a *Paul Whitfield Horn Professor at Texas Tech University* (TTU's highest academic distinction), Dr. Cooke exemplified excellence in both scholarship and education. His pioneering work on programming language theory and intelligent systems earned him the *IEEE Computer Society Technical Achievement Award* and the *NASA Exceptional Achievement Medal*, among numerous other honors.

Dr. Cooke's mentorship during my time at Texas Tech University had a lasting impact on my understanding of language theory and my philosophy of teaching. His passing is a profound reminder that even the brightest among us may face unseen struggles, and that seeking help is an act of strength. If you or someone you know is in distress, please reach out to TTU's Student Counseling Center or call the National Suicide Prevention Lifeline at **988**.

Backus-Naur Form (BNF) Rules for the Cooke Programming Language

In these projects you will be tasked with writing Lexical and Syntax Analyzers capable of parsing a source code file provided by the user written in a new programming language called "Cooke". The syntax for this language will be based upon the following grammar (in BNF):

```
P ::= S
S ::= V := E | cin >> V | cout << E | ++V | --V | for V in E .. E : S
      | if C : S M | if C : S M else : S | S ; S
M ::= elif C : S M | ε
C ::= C or A | A
A ::= A and R | R
R ::= not R | E < E | E > E | E == E | E <> E | E <= E | E >= E
E ::= T | E + T | E - T
T ::= F | T * F | T / F | T % F
F ::= (E) | I | L | V | ++V | --V
V ::= H X | _X
X ::= H X | D X | _X | ε
H ::= a | b | c | … | x | y | z
I ::= D | D I
L ::= . I | I . I
D ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Keep in mind that the *V*, *I*, and *L* grammar rules establish the formal rules for identifiers (*V*) and Integer literals (*I*) and Floating-Point Literals (*L*) respectively.

Lexemes for the Cooke Programming Language

Lexeme formation is guided using the BNF rules / grammar above. Your application should match each lexeme with its associated token. Invalid lexemes should be assigned to a token group called **UNKNOWN**. The following token names should be used to identify each valid lexeme:

Lexeme	Token
:=	ASSIGNMENT_OP
or	BOOLEAN_OR
and	BOOLEAN_AND
not	BOOLEAN_NOT
<	LESSER_OP
>	GREATER_OP
==	EQUAL_OP
<>	NEQUAL_OP
<=	LEQUAL_OP
>=	GEQUAL_OP
;	SEMICOLON
:	COLON

Lexeme	Token
(	LEFT_PAREN
)	RIGHT_PAREN
if	KEY_IF
elif	KEY_ELIF
else	KEY_ELSE
for	KEY_FOR
in	KEY_IN
cin	KEY_CIN
cout	KEY_COUT
>>	SHIFT_R_OP
<<	SHIFT_L_OP
..	RANGE_OP

Lexeme	Token
+	ADD_OP
-	SUB_OP
*	MULT_OP
/	DIV_OP
%	MOD_OP
++	INC_OP
--	DEC_OP
V	IDENT
I	INT_LITERAL
L	FLOAT_LITERAL

Project #1 – Lexical Analyzer

Problem

Develop a lexical analyzer in C or C++ that can identify lexemes and tokens found in a source code file provided by the user. Once the analyzer has identified the lexemes of the language and matched them to a token group, the program should print each lexeme / token pair to the screen.

The source code file provided by the user will be written in the new programming language called "Cooke" and is described on page 4 of this document.

Your program must accept the source code file as a required command-line argument and display an appropriate error message if the argument is not provided or the file does not exist. The command to run your application will look something like this:

Form: cooke_analyzer <path_to_source_file>
Example: cooke_analyzer test_file.dc

Project Requirements

Your solution must conform to the following rules:

1) Your solution is required to make use of the starting code provided to you on RaiderCanvas. You may modify or expand the provided starter code, but your final submission should maintain the same core structure, logic, and naming conventions.

2) Your solution should be able to use whitespace, tabs, end of line characters, and end of file characters as delimiters between lexemes, however your solution should ignore these characters and not report them as lexemes, nor should it **require** these characters to delimit lexemes of different types.
    a. Example: "f+15;"
        i. This line will generate the lexemes "f", '+', '15', and ";".

3) Your solution should print out "Cooke Analyzer :: R<#>" on the first line of output. The double colon "::" is required for correct grading of your submission.

4) Your solution must be tested to ensure compatibility with the GNU C/C++ compiler version 14.2.0.

5) Lexemes that do not match to a known token should be reported as an "UNKNOWN" token. This **should not** stop execution of your program or generate an error message.

6) Ensure your code compiles successfully on the HPCC using GCC 14.2.0. Submissions that do not compile in this environment will receive a score of 0%.
    a. It is not the responsibility of the grader to correct your code or makefile.

AI Usage Policy

Students are permitted to use artificial intelligence tools such as ChatGPT, GitHub Copilot, or similar systems to assist with the design, development, or debugging of their lexical analyzer. However, all AI use must be fully transparent and documented through an accompanying *"AI Reflection Report"* submitted separately for this project. The report will describe how AI tools were used, what parts of the code were personally implemented or modified, and include the full AI chat log or shareable link as an appendix.

Students remain fully responsible for understanding every part of their submitted code. At any point during or after grading, you may be asked to explain the structure, logic, or purpose of any section of your implementation. **Questions regarding your project and its specific BNF rules may appear on exams.** Code that you cannot clearly explain or that appears inconsistent with your documented AI usage may result in grade penalties or an academic integrity review.

For detailed report structure, formatting, and submission instructions, see Appendix A – AI Reflection Report Requirements on page 13.

Hints

1) Draw inspiration by looking at the lexical analyzer code discussed and distributed in class.
    a. Utilizing this code will not be considered plagiarism.
2) Start by focusing on writing the program in your usual C/C++ development environment.
3) Once your solution is correct, then work on testing it in Linux using the appropriate version of the GNU compiler (gcc).
4) If you are unsure of how to make use of command line arguments in C/C++, then please utilize one of the following resources:
    a. GeeksForGeeks - Command Line Arguments in C/C++
    b. TutorialsPoint - C/C++ Command Line Arguments
5) Test your solution manually using the provided test file and ensure your output is identical to the Example Execution below.

What to turn in to RaiderCanvas

A zip archive (.zip) containing the following files:

- makefile
    - A makefile for compiling your C/C++ file(s).
    - This makefile must work in the HPCC environment to compile your source code file and output an executable named cooke_analyzer.
- All source code files required to compile your program.
    - Failure to include all source code files will result in your program failing to compile – which will ensure you receive a 0% for correctness.
- Your detailed AI usage report as a PDF.
    - For details, see Appendix A – AI Reflection Report Requirements on page 13.
- **Ensure that your zip file does not contain extraneous files or directories.**

Example Execution

The example execution below was run on the HPCC clusters. It shows all the commands used to compile and execute my analyzer. Bolded text is text from the Linux OS, text in red are the commands I typed and executed, and the text in blue represents the output from each step.

```
login-20-25:/project_1$ make clean
rm -f cooke_analyzer

login-20-25:/project_1$ make
gcc -o cooke_analyzer Eric_Rees_R123456_Project1.c

login-20-25:/project_1$ ./cooke_analyzer example_1.dc
Cooke Analyzer :: R123456

f       IDENT
:=      ASSIGNMENT_OP
15      INT_LITERAL
;       SEMICOLON
i       IDENT
:=      ASSIGNMENT_OP
11.00   FLOAT_LITERAL
;       SEMICOLON
cin     KEY_CIN
>>      SHIFT_R_OP
n       IDENT
;       SEMICOLON
++      INC_OP
n       IDENT
;       SEMICOLON
if      KEY_IF
n       IDENT
<>      NEQUAL_OP
i       IDENT
:       COLON
cout    KEY_COUT
<<      SHIFT_L_OP
f       IDENT
+       ADD_OP
i       IDENT
```

Project #2 – Syntax Parser

Problem Statement

Using your lexical analyzer developed in Project #1 and the parser code provided in class as a guide, develop a **recursive descent** syntax parser in C or C++ that can validate the syntax of a source code file provided by the user.

The source code file provided by the user will be written in the new programming language called "Cooke" and is described on the first page of this document.

Your program must accept the source code file as a required command-line argument and display an appropriate error message if the argument is not provided or the file does not exist. The command to run your application will look something like this:

Form: cooke_parser <path_to_source_file>
Example: cooke_parser test_file.dc

Project Requirements

Your parser should read the given Cooke source code file then, using your lexical analyzer, identify lexemes/tokens one at a time to ensure they can be aligned with known BNF rules for the language. For this language, P::=S would be the root of the parse tree. Your parser may print out the steps it takes during execution if you wish (similar to the provided parser) but that is <u>not</u> required. The following is the only required output and exit codes from your parser:

1) Your solution is required to make use of the starting code provided to you on RaiderCanvas. Your final solution should still bear resemblance to this original code, even if it has been converted to C++.

2) Your solution is required to be implemented as a *recursive descent* parser. If your solution solves it using any other method or deviates from *recursive descent* then it will be considered incorrect and receive no "Correctness" points.

3) Your solution must print out "Cooke Parser :: R<#>" as the first line of output with <#> being replaced by your R#. The double colon "::" is required for correct grading of your submission.

4) If the provided user file is free of syntax errors:
    a. Your solution must print out "Syntax Validated" as the last line of output.
    b. Your solution must return with an exit code of 0.

5) If the provided user file contains syntax errors:
    a. Your solution must print out "Error encounter on line <num>: The next lexeme was <lexeme> and the next token was <token>"
        i. Where <num> is the line number where the error occurred (where the first line of a program is line 1)
        ii. Where <lexeme> is the lexeme that caused the problem (examples: "x", "<>")
        iii. Where <token> is the uppercase name of the token (examples: "IDENT", "UNKNOWN")
    b. Your solution must return with an exit code of 1.

6) If the user did not provide a file as input:
    a. Your solution must display an appropriate error message.
    b. Your solution must return with an exit code of 2.

7) If the user did provide a file as input but the file does not exist:
    a. Your solution must display an appropriate error message.
    b. Your solution must return with an exit code of 3.

8) If a user input file contains multiple syntax errors, your solution is only required to find and report the <u>first</u> syntax error.

9) Ensure your code compiles successfully on the HPCC using GCC 14.2.0. Submissions that do not compile in this environment will receive a score of 0%.
    a. It is not the responsibility of the grader to correct your code or makefile.

AI Usage Policy

Students are permitted to use artificial intelligence tools such as ChatGPT, GitHub Copilot, or similar systems to assist with the design, development, or debugging of their recursive-descent parser. However, all AI use must be fully transparent and documented through an accompanying *"AI Reflection Report"* submitted separately for this project. The report will describe how AI tools were used, what parts of the code were personally implemented or modified, and include the full AI chat log or shareable link as an appendix.

Students remain fully responsible for understanding every part of their submitted code. At any point during or after grading, you may be asked to explain the structure, logic, or purpose of any section of your implementation. **Questions regarding your project and its specific BNF rules may appear on exams.** Code that you cannot clearly explain or that appears inconsistent with your documented AI usage may result in grade penalties or an academic integrity review.

For detailed report structure, formatting, and submission instructions, see Appendix A – AI Reflection Report Requirements on page 13.

Hints

1) Draw inspiration by looking at the parser code discussed and distributed in class.
2) Keep in mind that because we are using a LL parser, the language's BNF rules **may** need to be updated.
3) Start by focusing on writing the program in your usual C/C++ development environment.
4) Once your solution is correct, then work on testing it in Linux using the appropriate version of the GNU compiler (gcc).

What to submit to RaiderCanvas

A zip archive (.zip) containing at least the following files:

- makefile
  - A makefile for compiling your C/C++ file(s).
  - This makefile must work in the HPCC environment to compile your source code file(s) and output an executable named cooke_parser.
- All source code files required to compile your program.
  - Failure to include all source code files will result in your program failing to compile – which will ensure you receive a 0% for correctness.
- Your detailed AI usage report as a PDF.
  - For details, see Appendix A – AI Reflection Report Requirements on page 13.

Ensure that your zip file does not contain extraneous files or directories.

Example Execution

The example execution below was run on the HPCC clusters. It shows all the commands used to compile and execute the parser for Cooke. Bolded text is text from the Linux OS, text in red are the commands I typed and executed, and the text in blue represents the output from each step.

```
login-20-25:/project_2$ make clean
rm -f cooke_parser

login-20-25:/project_2$ make
gcc -o cooke_parser Eric_Rees_R123456_Project_2.c

login-20-25:/project 2$ ./cooke_parser example_1.dc
Cooke Parser :: R123456

Syntax Validated

login-20-25:/project_2$ ./cooke_parser example_2.dc
Cooke Parser :: R123456

Error encounter on line 4:  The next lexeme was ++ and the next token was INC_OP
```

Optional Extra Credit

Project #2: Finding all syntax errors in a file (+15)

This extra credit opportunity requires your solution to detect and report multiple syntax errors in a file instead of stopping after the first error. Implementing this feature means making your parser more fault-tolerant and capable of "recovering" when invalid data is encountered.

Requirements for Extra Credit Consideration

1) **Baseline Requirement**
   a. Your code must score 80% or higher on the Project #2 grading script used for normal grading.
   b. If your solution does not meet this threshold, it will not be considered for extra credit.

2) **Error Detection Requirement**
   a. Your parser must correctly identify at least 30% of the known errors in a specially crafted test file containing multiple syntax errors.
   b. Failure to identify at least 30% of the known errors will result in no points being awarded.

3) **Grading Scale**
   a. The maximum extra credit is +15 points, but the actual points awarded will range from +0 to +15, depending on how effectively your parser handles the multiple-error test cases.
   b. The scoring mechanism will give points as follows:
      i. +1 point for each error identified correctly
      ii. -1 point for each error identified incorrectly (extraneous errors)
   c. The following table will inform you how to translate score into extra credit points:

Score	*Extra Credit*
<2 points	+0 Extra Credit
2 points	+3 Extra Credit
3 points	+6 Extra Credit
4 points	+9 Extra Credit
5 points	+12 Extra Credit
6 points	+15 Extra Credit

Examples:
If a student detects 3 errors correctly and has 1 false positive, their net score would be 2 points, earning +3 extra credit points.

If a student detects 2 errors correctly and has 3 false positives, their net score would be -1 points, earning 0 extra credit points.

4) **Single Parser**
   a. Your parser must be a single solution that handles both the original test cases (which contain only one syntax error) and the multiple-error test cases.
   b. You cannot submit two separate parsers—one for normal grading and another for extra credit.

Important Warning

Extra credit should only be attempted after you have successfully completed Project #2. Starting with the extra credit implementation before ensuring your core project works correctly may lead to significant issues, potentially preventing you from completing Project #2 properly. Focus on getting the required functionality working first before attempting this additional challenge.

Appendix A – AI Reflection Report Requirements

Purpose

To ensure transparency and academic integrity while allowing responsible use of artificial intelligence tools, each project requires an AI Reflection Report documenting your use of AI assistance and your understanding of the resulting work.

This report serves as evidence of authorship and process as it demonstrates *how* you developed your solution, *where* AI contributed, and *what* you personally accomplished.

Keep in mind this report is required even if you did not utilize AI. If you did not utilize AI then the Authorship Statement should be explicit in documenting that.

Submission Details

The AI Reflection Report is a required companion submission to each programming project. Its purpose is to document how you used AI tools during development and to demonstrate that you personally understand and authored the code you are submitting. This report allows for transparency, accountability, and fairness in the use of AI-assisted development while maintaining the academic integrity of the course.

- The AI Reflection Report must be submitted as part of your Project #1 and Project #2 submissions.
- For grade calculations, failure to submit this report will result in a 30 point deduction to each Project grade it is missing in.
- Reports must be submitted as PDF documents following the structure described below.

Report Layout

AI Reporting Guidelines

Because students may use different AI tools during development, your report must clearly document which tools were used and how their outputs were obtained. The following guidance explains how to record and submit AI interactions depending on the platform you used:

1. **ChatGPT, Claude, Gemini, or other conversational AI**
    a. Export or copy the full conversation that directly contributed to your project.
    b. You may either paste the complete transcript at the end of your report or include a shareable link if the platform supports it (e.g., ChatGPT's "Share" feature).
    c. Do not edit, summarize, or re-order the conversation. You may redact only unrelated or personal exchanges.
2. **GitHub Copilot or inline code assistants**
    a. Copilot and similar tools (e.g., Replit Ghostwriter, VS Code IntelliCode) do not maintain full chat transcripts.
    b. In your Authorship Statement, include a short section titled "Copilot Usage Summary" that lists:
        i. Which IDE/editor you used (e.g., VS Code with Copilot).

        ii. Which files or functions received Copilot suggestions.
        iii. Roughly what kinds of completions were accepted, modified, or discarded.
        iv. Any significant prompts or comments you typed that shaped those suggestions.
    c. If possible, include IDE log snippets or screenshots of Copilot suggestions as supporting evidence in your appendix.

3. **Multiple AI tools**
    a. If you used more than one AI system, list each separately with its corresponding chat link or summary.
    b. Make sure your documentation explains how each tool contributed to your final solution (design help, debugging, code generation, etc.).
4. **Privacy and formatting**
    a. You may redact personal information (names, unrelated conversations, API keys, etc.).
    b. All AI interactions must remain readable and complete enough for verification.

Section 1: Authorship Statement

This is the primary portion of your report and should be written in <u>your own words</u>. It should typically be 1–2 pages and written in a professional, reflective tone.

Your Authorship Statement should include the following elements:

- **Development Process Overview**
  Describe how you approached the project from start to finish. Summarize your coding workflow, testing, and debugging process. Discuss how AI fit into that workflow (e.g., design guidance, code skeletons, debugging advice, explanation of recursion, etc.).

- **Use of AI Tools**
  Identify which AI tools were used (e.g., ChatGPT, Copilot, Gemini) and explain specifically what they contributed (e.g., generating starter code, clarifying syntax errors, explaining a concept, etc.).

- **Your Personal Contributions**
  Explicitly identify what parts of the code or logic you personally wrote, rewrote, or substantially modified. Discuss *why* you changed or refined AI-generated code — for example, to meet formatting rules, fix logic errors, improve structure, or satisfy project constraints.

- **Reflection on AI Assistance**
  Briefly reflect on what AI did well or poorly in this assignment. Did it misunderstand the grammar? Did it require substantial human correction? What did you learn by debugging or refining AI output? This reflection is an opportunity to demonstrate your understanding, not to justify reliance on AI.

This section should read as a clear narrative of *your development process*, not a bullet-point summary.
By the end, the reader should have a confident understanding of your role in producing the final submission.
Once your report is structured according to the layout above, ensure it also satisfies the verification, authenticity, and professionalism requirements described below.

Section 2: AI Chat Logs

Following your Authorship Statement, include a clearly labeled Appendix containing the complete record of your AI interaction(s). This appendix serves as supporting documentation for your report and allows the instructor to verify your development process.

Your appendix should include either:

- The full, unedited text transcript of all AI conversations that contributed to your project; or
- A shareable link to the complete conversation(s) if your chosen platform (e.g., ChatGPT) supports it.

The appendix should be clearly formatted and easy to follow. If you used multiple AI sessions, include a brief heading before each one indicating its purpose (e.g., "Lexer Debugging Assistance," "Parser Grammar Clarification," or "Code Review Session.").

- The transcript or link must contain the full, unedited conversation beginning with your initial prompt and including all relevant exchanges that influenced your code.
    - You may remove unrelated or personal portions, but not any AI interactions tied to this assignment.
- The conversation must clearly correlate with your submitted code (naming conventions, structures, and logic).

By treating the appendix as an official part of your report, you are demonstrating full transparency in your use of AI while providing the necessary evidence to verify that your code and documentation are your own.

Report Requirements

Verification and Responsibility

- You are fully responsible for understanding and explaining all code that appears in your submission, regardless of AI involvement.
- During grading, you may be asked to explain the purpose or logic of any section of your code.
- Reports or submissions that do not match the AI log provided, or that show signs of unrelated or copied work, will be treated as academic dishonesty.

Authenticity and Correlation

- The AI conversation log must correspond directly to your submitted project code.
- Logs that appear edited, fabricated, or unrelated will result in a zero for the AI Reflection Report grade and may prompt further review.
- You must be able to reproduce, discuss, or modify your code independently if requested.

Professionalism and Format

- The report should be typed, professionally formatted, and clearly organized into the required sections.
- Use complete sentences, appropriate grammar, and a professional tone.
- The chat log appendix may be in plain text or PDF format attached at the end of the report.

Academic Integrity Reminder

- Using AI does not absolve you of authorship or responsibility.
- Your goal is to demonstrate comprehension, not outsourcing.
- Any report that fails to establish your authorship or omits a valid AI transcript will receive a zero for this portion and may be subject to academic integrity review.